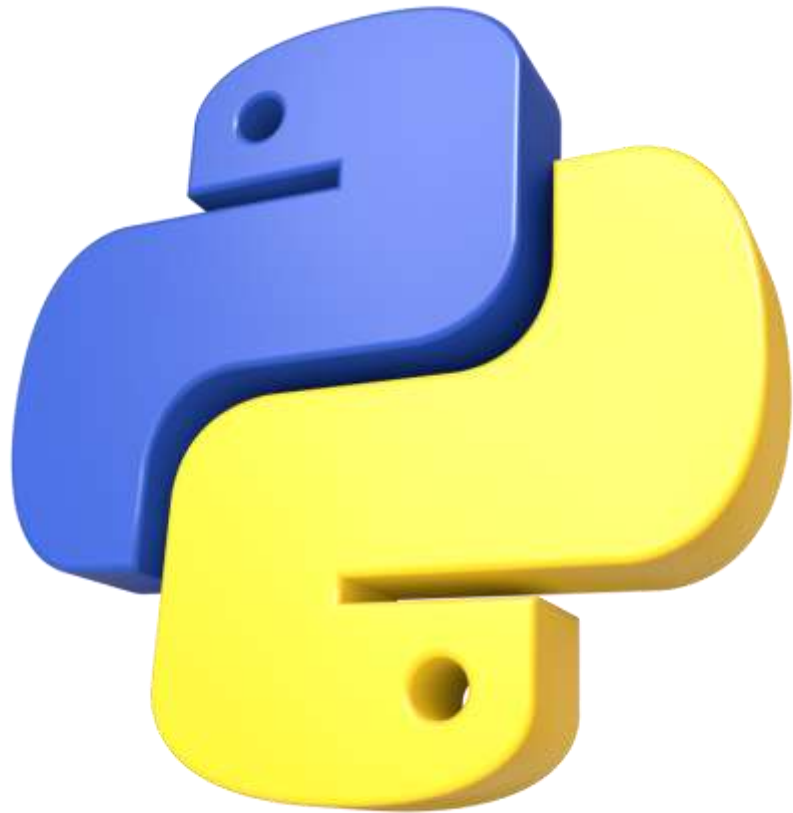




Aula 5

Prof. Gabriel Rodrigues e Wilson
Hissamu

Algoritmo e Programação

Temas desta aula

- Compreender estruturas condicionais
- Modelar decisões reais
- Aplicar lógica em problemas
- Exercícios

Python “Switch Case”

- Em muitas linguagens de programação, como C, C++ e Java, a estrutura “switch case” é uma ferramenta essencial para o controle de fluxo, permitindo que um valor seja verificado contra múltiplas condições de forma organizada e eficiente.
- Em **Python**, uma linguagem conhecida por sua simplicidade e legibilidade, não há uma estrutura “switch case” nativa.

Python “Switch Case”

- Essas linguagens possuem uma sintaxe específica para o “switch case”, onde você pode verificar um valor contra múltiplas condições de forma organizada e clara.
- No entanto, Python adota uma abordagem diferente para resolver esse problema, utilizando outras estruturas disponíveis na linguagem.

Python “Switch Case”

- Vantagens do uso do switch case
- Clareza e Legibilidade

Uma das principais vantagens do “switch case” é a clareza e a legibilidade que ele proporciona. Cada caso é claramente separado e identificado, o que facilita a leitura e compreensão do código. Isso é especialmente útil quando se lida com múltiplas condições que precisam ser verificadas contra um único valor.

Python “Switch Case”

- Vantagens do uso do switch case
- Eficiência de Execução

O “switch case” é geralmente mais eficiente do que uma cadeia de declarações if-else em muitas linguagens. Isso ocorre porque o compilador pode otimizar a estrutura “switch case” de forma mais eficaz, utilizando tabelas de salto ou outros mecanismos de otimização, resultando em um tempo de execução mais rápido.

Python “Switch Case”

- Vantagens do uso do switch case
- Manutenção Facilitada

A estrutura clara e organizada do “switch case” facilita a manutenção do código. Adicionar, remover ou modificar casos individuais é uma tarefa simples, reduzindo a chance de erros que podem ocorrer ao manipular longas cadeias de declarações if-else.

Python “Switch Case”

- Vantagens do uso do switch case
- Redução da Redundância

Ao usar “switch case”, é possível reduzir a redundância no código, agrupando casos que executam a mesma lógica. Em linguagens que suportam essa estrutura, múltiplos casos podem compartilhar a mesma instrução sem a necessidade de duplicar o código.

Python “Switch Case”

- Vantagens do uso do switch case
- Escalabilidade

A estrutura “switch case” é naturalmente escalável, permitindo que desenvolvedores adicionem novos casos conforme necessário, sem comprometer a legibilidade do código. Isso é particularmente útil em aplicações onde o número de condições pode crescer com o tempo.

Sintaxe

- Embora Python não tenha uma estrutura “switch case” nativa, ele oferece várias alternativas para alcançar o mesmo efeito.

Sintaxe

- Usando If-Elif-Else

A abordagem mais direta é usar uma cadeia de declarações if-elif-else:

- **if**: Avalia a primeira condição.
- **elif**: Avalia condições adicionais, se as anteriores forem falsas.
- **else**: Opcional, define o caso padrão se nenhuma das condições anteriores for verdadeira.

Sintaxe

- Usando If-Elif-Else

A abordagem mais direta é usar uma cadeia de declarações if-elif-else:

- **if**: Avalia a primeira condição.
- **elif**: Avalia condições adicionais, se as anteriores forem falsas.
- **else**: Opcional, define o caso padrão se nenhuma das condições anteriores for verdadeira.

Sintaxe

- Usando Dicionários

Outra abordagem eficiente é usar dicionários para mapear valores a funções ou resultados:

- **Dicionário (cases)**: Mapeia valores (chaves) a funções ou resultados.
- **get(value, lambda: “Caso padrão”)**: Retorna a função correspondente ao valor ou uma função padrão.

Sintaxe

- Usando Match-Case (Python 3.10+)
- A partir do Python 3.10, a estrutura match-case foi introduzida, fornecendo uma sintaxe mais elegante e poderosa:
- **match value**: Inicia a estrutura de correspondência com o valor a ser avaliado.
- **case valor**: Define um caso específico a ser comparado com o valor.
- **case _**: Define o caso padrão, correspondendo a qualquer valor não capturado pelos casos anteriores.

Quando utilizar?

- **Verificação de Múltiplos Casos Simples**

Quando você precisa verificar um valor contra várias condições discretas, o uso de `if-elif-else` ou `match-case` pode ser ideal.

Essa situação é comum em casos onde uma variável pode ter um conjunto limitado de valores conhecidos, e cada valor requer um tratamento específico.

Quando utilizar?

- **Substituição de Estruturas Longas de If-Elif-Else**

Quando você se encontra com uma longa cadeia de if-elif-else, especialmente se ela se torna difícil de ler e manter, o uso de um dicionário para mapear valores a funções pode simplificar significativamente o código.

Essa abordagem é útil para evitar duplicação de código e melhorar a legibilidade.

Quando utilizar?

- **Implementação de Máquinas de Estado**

Máquinas de estado, que são usadas para modelar o comportamento de sistemas onde a saída depende de uma sequência de eventos, podem se beneficiar do uso de dicionários ou match-case.

Cada estado e transição pode ser representado como um caso, facilitando a implementação e manutenção da lógica complexa.

Quando utilizar?

- **Manutenção e Escalabilidade do Código**

Quando o código precisa ser escalável e facilmente mantido, a escolha de usar match-case ou dicionários pode ser vantajosa.

Adicionar novos casos ou modificar os existentes torna-se uma tarefa mais organizada e menos propensa a erros, especialmente quando comparada com a modificação de longas cadeias de if-elif-else.

Como usar o Switch Case no Python em diferentes métodos

- **If-Elif-Else**

Use quando você tem um número pequeno de casos e deseja manter a simplicidade. É fácil de entender e implementar, mas pode se tornar confuso em muitos casos.

Como usar o Switch Case no Python em diferentes métodos

- **If-Elif-Else**

```
def switch_case_if_elif_else(value):  
  
    if value == 1:  
  
        return "Caso 1"  
  
    elif value == 2:  
  
        return "Caso 2"  
  
    elif value == 3:  
  
        return "Caso 3"  
  
    else:  
  
        return "Caso padrão"  
  
  
resultado = switch_case_if_elif_else(2)  
  
print(resultado)  # Saída: Caso 2
```

Como usar o Switch Case no Python em diferentes métodos

- **If-Elif-Else**

Vantagens: Simplicidade e clareza para um pequeno número de casos.

Desvantagens: Pode se tornar longo e difícil de manter com muitos casos.

Como usar o Switch Case no Python em diferentes métodos

- **Dicionários**

Ideal para reduzir o código redundante e melhorar a manutenção quando você tem muitas condições discretas a serem verificadas.

É eficiente e elegante, mas requer compreensão de funções como valores.

Como usar o Switch Case no Python em diferentes métodos

- **Dicionários**

```
def caso1():  
  
    return "Caso 1"  
  
def caso2():  
  
    return "Caso 2"  
  
def caso3():  
  
    return "Caso 3"  
  
def switch_case_dicionario(value):  
  
    cases = {  
  
        1: caso1,  
  
        2: caso2,  
  
        3: caso3  
  
    }  
  
    resultado = switch_case_dicionario(2)  
  
    print(resultado) # Saída: Caso 2
```

Como usar o Switch Case no Python em diferentes métodos

- **Dicionários**

Vantagens: Redução de redundância e melhoria da legibilidade.

Desvantagens: Pode ser menos intuitivo para novatos que não estão familiarizados com funções como valores.

Como usar o Switch Case no Python em diferentes métodos

- **Match-Case**

A escolha preferida se você estiver usando Python 3.10 ou superior. Proporciona uma maneira moderna e poderosa de lidar com múltiplos casos, com suporte adicional para correspondência de padrões.

Como usar o Switch Case no Python em diferentes métodos

- **Match-Case**

```
def switch_case_match_case(value):  
  
    match value:  
  
        case 1:  
  
            return "Caso 1"  
  
        case 2:  
  
            return "Caso 2"  
  
        case 3:  
  
            return "Caso 3"  
  
        case _:  
  
            return "Caso padrão"  
  
resultado = switch_case_match_case(2)  
  
print(resultado)  # Saída: Caso 2
```

Como usar o Switch Case no Python em diferentes métodos

- **Match-Case**
- Escolha uma opção (A, B, C)
- Mostre a opção selecionada

Como usar o Switch Case no Python em diferentes métodos

- **Match-Case**

- *# Exemplo prático de Match Case* `opcao = input("Escolha uma opção (A, B, C): ").upper()` `match opcao:` `case 'A': print("Você selecionou a opção A.")` `case 'B': print("Você selecionou a opção B.")` `case 'C': print("Você selecionou a opção C.")` `case _:` `print("Opção inválida.")` *# O '_' funciona como 'else' (padrão)*
- O método `.upper()` em Python serve para converter todos os caracteres minúsculos de uma string em maiúsculos.

Como usar o Switch Case no Python em diferentes métodos

- **Match-Case**

Exercício 16 – Menu de opções

Peça ao usuário para digitar uma opção:

1 – Olá

2 – Bem-vindo

3 – Python

Mostre a mensagem correspondente à opção escolhida.

Caso a opção não exista, mostre `**"Opção inválida"**`.

Como usar o Switch Case no Python em diferentes métodos

- **Match-Case**

Exercício 16 – Menu de opções

```
opcao = input("Digite uma opção:\n1 – Olá\n2 – Bem-vindo\n3 – Python\n")
```

```
match opcao:
    case "1":
        print("Olá")
    case "2":
        print("Bem-vindo")
    case "3":
        print("Python")
    case _:
        print("Opção inválida")
```